JAILEDDOTFUN ENHANCEMENT 2
Security Review



FEBRUARY, 2026

www.chaindefenders.xyz
https://x.com/DefendersAudits

Lead Auditors



PeterSR



0x539.eth

Table of Contents

1. Protocol Summary
2. Disclaimer
3. Risk Classification
4. Audit Details

- Commit Hash
- Scope
- Roles

5. Executive Summary

- Issues Found
- Issues Statuses
- Security Grade

6. Findings

- Low

Protocol Summary

A Solana-based pickaxe training and exploration game. It blends NFT-style assets, STEPN-like exploration loops, and a Token-2022 economy with JAIL. On-chain handles ownership and verification; off-chain handles computation and player experience.

This update introduces several structural improvements to the breeding flow, treasury handling, signature domain separation, and token initialization logic. A dedicated BreedingTreasury PDA has been added to temporarily hold SOL during the breeding process, ensuring that funds are escrowed safely and can be refunded if a breeding attempt is canceled, instead of being transferred directly to a treasury or admin account before completion. The signed breeding cost payload

has been extended to include a `sol_cost` field, which is now part of both the admin-signed message and the `BreedingAttempt` account state, cryptographically binding the SOL price to the signed data and preventing mismatches or tampering. A new `breeding_treasury` field has been added to `GameConfig` to define the final payout destination for completed breeding fees, allowing flexible treasury management without hardcoding addresses. Additionally, a `JAIL_INITIAL_SUPPLY` constant of 10,000,000 tokens has been introduced, and during initialization this amount is minted directly to the treasury in a controlled and deterministic manner. To prevent double minting, a `jail_initialized` guard has been added to ensure the initialization logic can only execute once. An admin-only `update_breeding_treasury` instruction has also been implemented to allow secure updates of the treasury address stored in `GameConfig`. Finally, the domain message format used for signed payloads has been shortened by incorporating the first eight bytes of the SHA-256 hash of the program tag, reducing message size while preserving proper domain separation and signature integrity.

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Impact/Likelihood	High	Medium	Low
High	High	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Audit Details

Commit Hash

f08f3b35c1a9a929caa54e47cf81acfdba91835b

Scope

Id	Files in scope
1	./constants.rs
2	./errors.rs
3	./heap.rs
4	./lib.rs
5	./state.rs
6	./instructions/mod.rs
7	./instructions/admin/admin.rs
8	./instructions/admin/burn.rs
9	./instructions/admin/mod.rs
10	./instructions/boxes/mod.rs
11	./instructions/boxes/mystery_rewards.rs
12	./instructions/boxes/pickaxe_boxes.rs
13	./instructions/breeding/mod.rs
14	./instructions/breeding/pickaxe.rs
15	./instructions/breeding/vrf.rs
16	./instructions/equipment/gem_inventory.rs
17	./instructions/equipment/gem_level.rs
18	./instructions/equipment/gems.rs
19	./instructions/equipment/initialize.rs
20	./instructions/equipment/mod.rs
21	./instructions/equipment/pickaxe_inventory.rs
22	./instructions/equipment/pickaxe_repair.rs
23	./instructions/exploration/loadout.rs
24	./instructions/exploration/mod.rs
25	./instructions/exploration/rewards.rs

Id	Files in scope
26	./instructions/exploration/start.rs
27	./instructions/marketplace/marketplace.rs
28	./instructions/marketplace/mod.rs
29	./instructions/token/jail_token.rs
30	./instructions/token/mod.rs
31	./utils/domain.rs
32	./utils/mod.rs
33	./utils/shared.rs
34	./utils/signature.rs
35	./utils/vrf_helpers.rs

Note: The scope of the audit was PR 57.

Roles

Id	Roles
1	Admin
2	Operator
3	User

Executive Summary

Issues Found

Severity	Count	Description
High	0	Critical vulnerabilities
Medium	0	Significant risks
Low	3	Minor issues with low impact
Informational	0	Best practices or suggestions
Gas	0	Optimization opportunities

Issues Statuses

Id	Title	Status
L-01	‘jail_initialized‘ Flag Is Not Always Validated When Jail Mint Is Used	Fixed
L-02	‘breeding_treasury‘ Account In ‘CancelPickaxeBreeding‘ Struct Should Already Exist	Fixed
L-03	‘BreedingTreasury‘ PDA Is Not Made Rent-exempt	Fixed

Security Grade



Based on the audit findings and code quality, the overall protocol security grade is 98.50 / 100.00. This reflects an excellent security posture with almost no room for improvement.

Findings

Low

Low 01 `jail_initialized` Flag Is Not Always Validated When Jail Mint Is Used

Location

mystery_rewards.rs

pickaxe.rs

gem_level.rs

pickaxe_repair.rs

Description

Several instruction handlers that interact with the JAIL token mint (`open_mystery_box` in `mystery_rewards.rs`, `request_pickaxe_breeding` in `pickaxe.rs`, `request_gem_level_up` in `gem_level.rs`, `cancel_gem_level_up` in `gem_level.rs` and `repair_pickaxes` in `pickaxe_repair.rs`) load the `GameConfig` account without verifying that the JAIL token has been properly initialized.

The `GameConfig` struct contains a `jail_initialized` flag that indicates whether the JAIL mint and related infrastructure have been set up. However, the account constraints in these instructions do not require this flag to be `true`, meaning instructions can proceed even if:

- The JAIL mint has not been created
- The mint authority PDA has not been established

This creates a scenario where users can submit valid transactions that fail due to missing or incomplete setup, leading to redundant transaction attempts, wasted transaction fees, and a poor user experience.

Recommendation

Add explicit constraints to the `game_config` account in all instructions that depend on JAIL token operations to verify initialization:

```
#[account(
    seeds = [crate::constants::CONFIG_SEED, game_config.config_seed.as_ref()],
    bump = game_config.bump,
    constraint = game_config.jail_initialized == true @ GameError::
    JailTokenNotInitialized
)]
pub game_config: Account<'info, GameConfig>,
```

This ensures that:

1. Only initialized game instances can execute JAIL-dependent operations
2. Clear, deterministic errors are returned early if preconditions are not met
3. Transaction failures occur during instruction validation rather than during execution, reducing wasted fees
4. The intent of the `jail_initialized` flag is properly enforced at the runtime level

Apply this constraint consistently across all affected instruction handlers: `open_mystery_box` in `mystery_rewards.rs`, `request_pickaxe_breeding` in `pickaxe.rs`, `request_gem_level_up` in `gem_level.rs`, `cancel_gem_level_up` in `gem_level.rs` and `repair_pickaxes` in `pickaxe_repair.rs`

Status

Fixed

Low 02 `breeding_treasury` Account In `CancelPickaxeBreeding` Struct Should Already Exist

Location

pickaxe.rs

Description

The `breeding_treasury` account in the `CancelPickaxeBreeding` struct is defined with the `init_if_needed` constraint, which allows the account to be created if it doesn't already exist. However, this is problematic because:

1. Invariant Violation: A breeding treasury should have been created and funded during the initial `request_pickaxe_breeding` call. If a cancel operation is being executed, the treasury must already exist to hold the SOL that needs to be refunded back to the player.
2. Unnecessary Cost: If the treasury somehow doesn't exist, the operator pays for account creation during cancellation, which shouldn't be necessary since the account should already be initialized.

Recommendation

Remove the `init_if_needed` constraint and replace it with a strict account existence requirement. Mark the account as mutable to indicate that its state (lamports) will be modified during the refund operation:

```
#[account(
    mut,
    seeds = [constants::BREEDING_TREASURY_SEED],
    bump = breeding_treasury.bump
)]
pub breeding_treasury: Account<'info, BreedingTreasury>,
```

This change:

- Enforces correctness: The treasury must exist before cancellation is allowed, ensuring breeding state consistency

- Fails fast: Missing treasury accounts produce clear deserialization errors rather than silent account creation
- Reduces waste: Eliminates unnecessary account initialization costs

Status

Fixed

Low 03 `BreedingTreasury` PDA Is Not Made Rent-exempt

Location

pickaxe.rs

Description

The `BreedingTreasury` PDA is initialized via `init_if_needed` with the exact required space (`BreedingTreasury::LEN`), but no lamports are transferred to cover the rent-exempt minimum balance. As a result, the account starts (and usually remains) below the rent-exempt threshold after creation.

This issue is compounded by the use of direct lamport manipulation in `transfer_lamports_if_needed`:

```
let mut from_lamports = from.try_borrow_mut_lamports()?;
require!(**from_lamports >= amount, GameError::InsufficientTreasuryFunds);
**from_lamports -= amount;

let mut to_lamports = to.try_borrow_mut_lamports()?;
**to_lamports += amount;
```

instead of using `system_program::transfer` CPI calls.

Consequences:

- The treasury PDA is almost immediately non-rent-exempt, once lamports fall below the rent-exempt threshold.
- Repeated small `sol_cost` breeding attempts accelerate the risk.
- Direct lamport mutation bypasses runtime transfer validation, creates a less auditable trail, and increases the chance of subtle bugs (underflow, rent interaction issues, etc.) in future changes.

Recommendation

Do one of the following:

1. Immediately after `init_if_needed` (or on every use if safer), top up the treasury to rent-exempt level using a CPI transfer from the payer:

```
use anchor_lang::system_program::{transfer, Transfer};
use anchor_lang::solana_program::rent::Rent;

let treasury_info = ctx.accounts.breeding_treasury.to_account_info();
let rent = Rent::get()?;
let exempt_balance = rent.minimum_balance(BreedingTreasury::LEN);

if treasury_info.lamports() < exempt_balance {
    let top_up = exempt_balance.saturating_sub(treasury_info.lamports());

    transfer(
        CpiContext::new(
            ctx.accounts.system_program.to_account_info(),
            Transfer {
                from: ctx.accounts.payer.to_account_info(),
                to: treasury_info.clone(),
            },
        ),
        top_up,
    )?;
}
```

2. Replace all direct lamport mutations with proper `system_program::transfer` CPI calls, signing with PDA seeds when transferring *from* the treasury:

```
pub fn transfer_to_treasury<'info>(
    system_program: &Program<'info, System>,
    payer: &Signer<'info>,
    treasury: &Account<'info, BreedingTreasury>,
    amount: u64,
) -> Result<()> {
    if amount == 0 { return Ok(()); }
    transfer(
        CpiContext::new(
            system_program.to_account_info(),
            Transfer {
                from: payer.to_account_info(),
                to: treasury.to_account_info(),
            },
        ),
        amount,
```

```
    )
}

pub fn payout_from_treasury<'info>(
    system_program: &Program<'info, System>,
    treasury: &Account<'info, BreedingTreasury>,
    destination: &AccountInfo<'info>,
    amount: u64,
) -> Result<()> {
    if amount == 0 { return Ok(()); }
    let seeds = &[constants::BREEDING_TREASURY_SEED, &[treasury.bump]];
    transfer(
        CpiContext::new_with_signer(
            system_program.to_account_info(),
            Transfer {
                from: treasury.to_account_info(),
                to: destination.clone(),
            },
            &[&seeds[..]],
        ),
        amount,
    )
}
```

Status

Fixed